Algorithm for File Updates in Python

Portfolio Activity — Python Automation for Access Control Management

Prepared by: G S Krishna Chandra | Google Cybersecurity Certificate — Course 7 (Automate Cybersecurity Tasks with Python)

Project Description

As a security professional at a health care company, part of my role involves managing access to restricted content that contains personal patient records. Access is controlled using an IP address allow list stored in a file called `allow_list.txt`. When employees no longer require access, their IP addresses are added to a separate remove list. In this project, I developed a Python algorithm that automates the process of reading the allow list, comparing it against the remove list, removing any matching IP addresses, and writing the updated list back to the file — ensuring that access controls remain current without manual intervention.

Open the File That Contains the Allow List

The first step is to assign the filename to a variable and open the file using a `with` statement:

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
```

The `import_file` variable stores the string `"allow_list.txt"` — the name of the file to be opened. The `with` statement is used alongside the `open()` function to open the file. The `open()` function takes two arguments: the filename and the mode — `"r"` indicates read mode. The `with` keyword ensures that the file is automatically closed once the block of code inside it finishes executing, even if an error occurs. The `as file` clause assigns the opened file object to the variable `file` so it can be referenced within the `with` block.

Read the File Contents

Inside the `with` block, the file contents are read into a string variable:

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
    ip_addresses = file.read()
```

The `.read()` method is applied to the `file` object. It reads the entire contents of the file and returns them as a single string. This string is stored in the variable `ip_addresses`. Storing the contents as a string is an intermediate step — it allows the data to be manipulated further using string methods before being converted into a list format.

Convert the String into a List

The string of IP addresses is converted into a list using the `.split()` method:

```
ip_addresses = ip_addresses.split()
```

The `.split()` method is a Python string method that splits a string into a list based on whitespace (spaces, newlines, or tabs) by default. Since each IP address in the file is on its own line, `.split()` separates them into individual elements in a list. Converting the string to a list is necessary because the `.remove()` method used in the next step only works on list objects, not strings. The result is reassigned to the same variable `ip_addresses`.

Iterate Through the Remove List

A `for` loop is used to iterate through each element in the remove list:

```
for element in remove_list:
```

The `for` keyword begins the loop. `element` is the loop variable — it takes the value of each item in `remove_list` on each iteration. The `in` keyword specifies the iterable object, which is `remove_list`. On each pass through the loop, `element` holds one IP address from the remove list, which will be used in the conditional check within the loop body.

Remove IP Addresses That Are on the Remove List

Inside the loop body, a conditional checks whether the IP address exists in the allow list before removing it:

```
for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)
```

The `if` statement checks whether the current `element` (an IP address from the remove list) is present in the `ip_addresses` list using the `in` operator. This conditional is important — without it, calling `.remove()` on an element that doesn't exist in the list would raise a `ValueError`. If the condition is true, the `.remove()` method is applied to `ip_addresses`, removing the first occurrence of `element` from the list. Applying `.remove()` in this way is reliable because there are no duplicate IP addresses in the `ip_addresses` list — each IP address appears only once, so `.remove()` will always remove exactly the intended entry.

Update the File with the Revised List of IP Addresses

The updated list is converted back to a string and written to the file:

```
ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

The `.join()` method is applied to the newline string `"\n"`, which acts as the separator between elements when the list is reassembled into a string. This places each IP address on its own line in the output, preserving the original file format. The result is reassigned to `ip_addresses`. A second `with` statement then opens the file in write mode (`"w"`), which overwrites the existing file contents. The `.write()` method writes the updated `ip_addresses` string to the file, completing the algorithm.

Complete Algorithm

```
import_file = "allow_list.txt"

remove_list = ["192.168.97.105", "192.168.158.170",
"192.168.201.40", "192.168.58.57"]

with open(import_file, "r") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)

ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

Summary

In this project, I developed a Python algorithm to automate IP address access control management for a health care company's restricted subnetwork. The algorithm opens the `allow_list.txt` file using a `with` statement and the `open()` function in read mode, reads its contents into a string using `.read()`, and converts that string into a list using `.split()` so individual IP addresses can be manipulated. A `for` loop then iterates through each IP address in the remove list, and a conditional statement checks whether each address exists in the allow list before applying `.remove()` to delete it — preventing errors from attempting to remove addresses that aren't present. Finally, the updated list is converted back to a newline-separated string using `.join()` and written back to the file using a second `with` statement in write mode with the `.write()` method. This algorithm demonstrates how Python can automate repetitive access control tasks, reducing human error and ensuring security policies are applied consistently.